

EngSci 721

Inverse Problems and Learning From Data

Oliver Maclaren (oliver.maclaren@auckland.ac.nz)

Module overview

Inverse Problems and Learning From Data (*Oliver Maclaren*)

[~14 lectures/3 tutorials]

1. *Basic concepts* [5 lectures + 1 Tutorial]

Forward vs inverse problems. Well-posed vs ill-posed problems. Algebra and calculus of inverse problems (left and right inverses, generalised and pseudo inverses, resolution operators, matrix calculus). Representing higher dimensional problems (image data etc).

2. *Instability and regularisation in linear and nonlinear problems* [6 lectures + 1 Tutorial]

Instability and related issues for generalised inverses. Introduction to regularisation and trade-offs. Tikhonov regularisation. Higher-order Tikhonov regularisation. Sparsity and regularisation using different norms. Truncated singular value decomposition. Iterative regularisation, including stochastic/mini-batch gradient descent.

3. *Further topics* [3 lectures + 1 Tutorial]

Regularisation parameter choice, including statistical and machine learning views of regularisation. Confidence sets for linear and nonlinear models. Physics-informed machine learning and neural networks.

Lecture 12: Regularisation parameter choice

Topics:

- Discrepancy principles
- Cross-validation
- Further reading

Eng Sci 721 : Lecture 12.

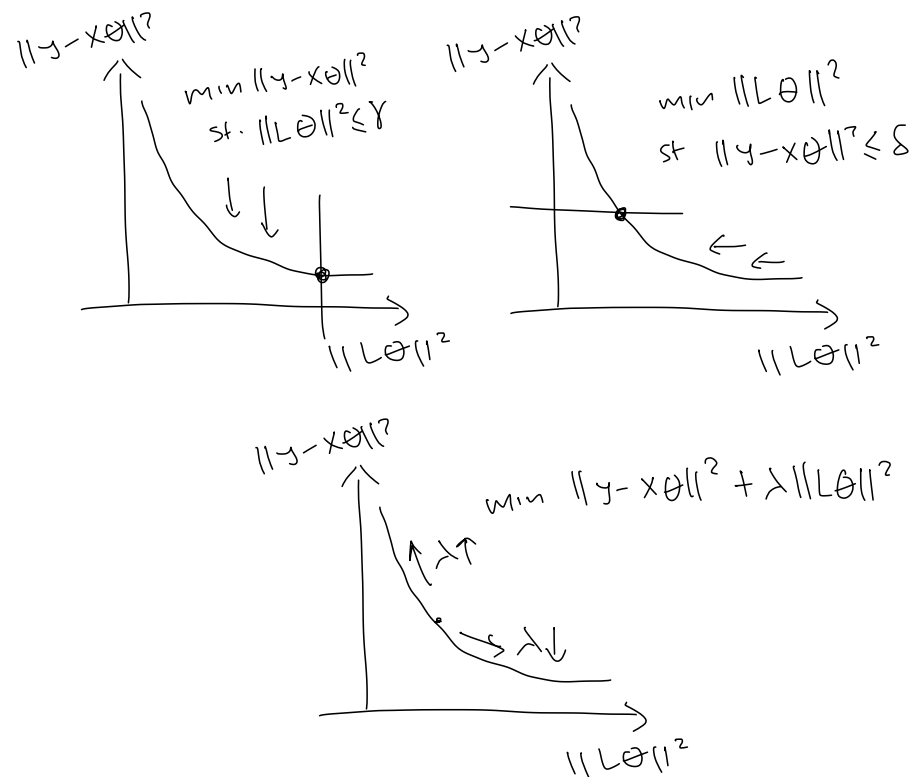
o Regularisation parameter choice methods

- Discrepancy approaches
- Cross-validation
- Further reading

Note: key takeaways are just
general ideas, not details

recall three ways to think about

trade-off curves:



These are 'equivalent' in the sense
that they give the same solⁿ for
appropriate choice of λ, δ, γ

Discrepancy principles

The general idea of a discrepancy principle is to solve eg

$$\begin{cases} \min \|L\theta\|^2 & \text{model norm term} \\ \text{st. } \|y - X\theta\|^2 \leq S & \text{data norm term} \end{cases}$$

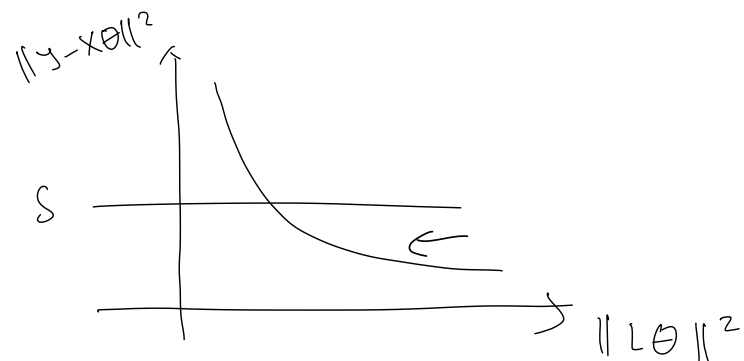
for some choice of S

Assuming the constraint is binding,
this means we enforce $\|y - X\theta\|^2 = S$
for some S .

Different methods of choosing S
exist. Here is one that has
a nice interpretation (IMO!)

→ 'simplest adequate/acceptable
fitting model'

Consider:



In some cases we may have info on
 S , eg if we have a statistical
model $y_i = x_i \theta^{\text{true}} + \epsilon_i$ (capital s : RVs)
where the distribution of ϵ_i is
known, eg

$$\epsilon_i \sim N(0, \sigma^2), \quad E = [\epsilon_1, \epsilon_2, \dots]^T$$

→ although we don't know ϵ_i , we hence
know the distribution of $\|E\|^2$
from the distribution of ϵ_i →

Chi-squared distribution

Give a series of standard normal random variables, i.e. $E_i \sim N(0, 1)$
↑ 'standard'

The distribution of

$$\left\{ Q = \sum_{i=1}^k E_i^2 \right\} \text{ i.e. sum of squared errors}$$

is called the chi-square / chi-squared distribution with k 'degrees of freedom'

Write $Q \sim \chi^2_k$ ↑ number of errors.

If we assume $y_i = x_i \theta + \varepsilon_i$ for $i = 1, \dots, m$

where $\varepsilon_i = N(0, \sigma^2)$, then

$$\left\{ \frac{E_i = y_i - x_i \theta}{\sigma} \right\} \text{ have dist. } N(0, 1) \text{ so}$$

↑
standard deviation

$$\Rightarrow \left\{ \frac{\|y - X\theta\|^2}{\sigma^2} \sim \chi_m \right\}$$

y, X random vars here

Hypothesis test inversion & confidence intervals

We don't know θ^{true} but we can test

$$H_0: \theta^{\text{true}} = \theta$$

for any θ !

Logic: • Suppose data is/are a 'typical' realisation, y^{obs} , such that the realised sum of squared errors lies in the 95% region of $\frac{1}{\sigma^2} \|y - X\theta^{\text{true}}\|^2$

(Fisher)

• If $\frac{1}{\sigma^2} \|y^{\text{obs}} - X\theta\|^2$ does not lie in the 95% region of $\frac{1}{\sigma^2} \|y - X\theta\|^2$ then θ cannot be equal to θ^{true} (given above)
→ rule it out!

→ otherwise call non-rejected
↳ not the same as 'accepted' as unique answer: ill-posed!

• Can show set of all non-rejected parameters forms a confidence set for θ^{true}

↳ i.e. a procedure for constructing random sets that will trap true value 95% of the time

Side track

Ru will cover Bayes... just a note:

Beware of Bayesian descriptions
of frequentist inference & vice-versa
... often dogmatic, often wrong!

Also applies to Bayesian descriptions of
Bayes & freq. description of freq.!

Side track: Frequentist Statistics: trust the process

Provides methods of evaluating the performance of
estimation methods, under repeated use in a
'noisy' environment (noisy observations/realisations of
random variables), regardless of where the
methods come from... Eg can do:

'frequentist performance of
Bayesian methods'!

Methods?

An estimator is a method (function)
that provides estimates of an estimand
(parameter/target) given observations

→ eg the generalised inverse is an estimator:

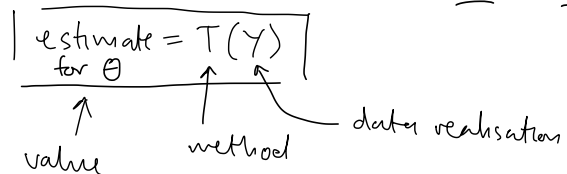
$$\begin{array}{c} \theta^+ = A^+ y \\ \uparrow \quad \quad \quad \nwarrow \\ \text{estimate of } \theta \quad \quad \text{estimator} \end{array} \quad \begin{array}{l} \swarrow \\ \text{data (realisation of} \\ \text{random var)} \end{array}$$

Where:

$$\begin{array}{c} A \leftarrow \text{forward model (for mean)} \\ \theta \xleftrightarrow{\quad} y \leftarrow \text{data (or sample)} \\ \uparrow \quad \quad \quad \nwarrow \\ \text{parameter} \quad \quad \text{estimator} \end{array}$$

Frequentist statistics as a two player game.

- nature chooses fixed θ & generates random data $Y \sim P(Y; \theta)$
- you come up with a 'guessing method' T (estimator) that gives an estimate of θ given any realisation of Y (i.e. you see data, not θ)



→ The estimate is random because Y (data) is random, even if θ fixed (but unknown)

Here:

- Probability applies to
 - 'actual variation' in nature (aleatoric)
 - The estimates that the guessing method 'spits out' (since estimator is a function of the data, $T(Y)$)
- Does not apply to 'fixed but unknown' things (e.g. θ)
 - Not rep. of personal uncertainty (epistemics)

Side track: Types of estimate: Many!

Point estimate: 'single best guess' $T(Y) = \theta_{\text{est}} \in \text{parameter space}$ } eg max likelihood estimate

Interval/set estimate: a set (eg interval) of parameter values indicating 'good', 'plausible' etc range of parameter values/estimates, i.e. $T(Y) = S_{\theta_{\text{est}}} \in \text{param space}$
↳ eg confidence interval.

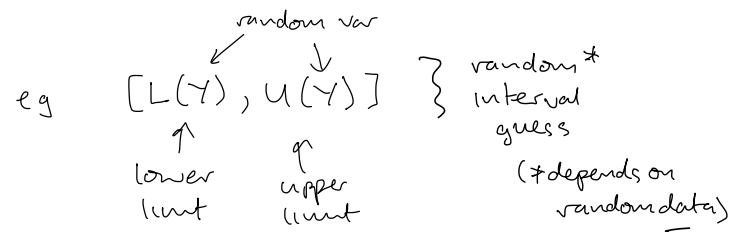
Function or distribution estimate:

(* more common for Bayes)

a distribution or function over parameter values, indicating 'good', 'plausible' etc ranges of parameter values/estimates.
→ likelihood function
→ posterior distribution
→ confidence distribution etc.

Sidetrack: Confidence intervals?

A confidence interval procedure is a method of guessing intervals given data realisations:



The method has a given coverage probability (eg 95% prob) which is the prob. of containing the true (usually fixed but unknown) value θ , ie $P([L(Y), U(Y)] \ni \theta; \theta)$, under repeated samples/realisations of Y .

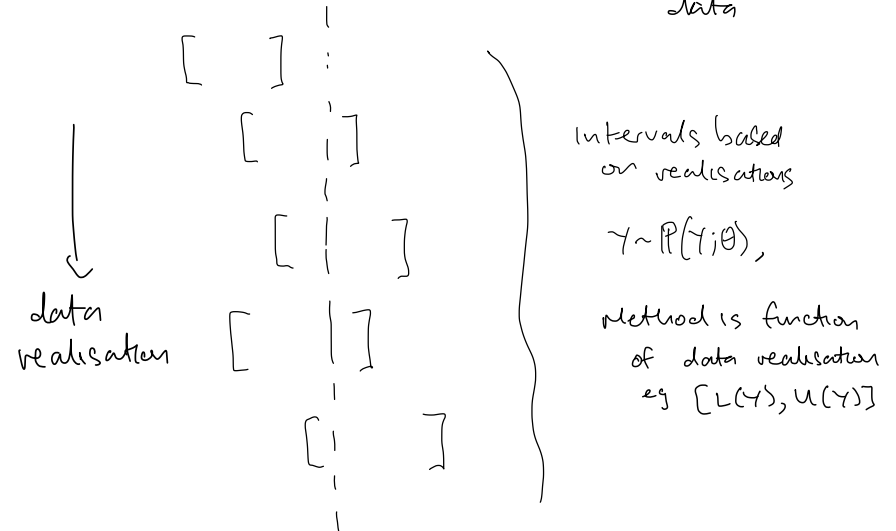
A given interval realisation either contains the true value or doesn't. Not 95% prob.

→ compare $P(\overset{\text{random var}}{Y}=3)$ to $P(\overset{\text{realisation}}{4}=3)$ } doesn't make sense

eg $P_Y([L(Y), U(Y)] \ni \theta; \theta) \checkmark$
 $P_Y([3, 5] \ni \theta; \theta) \times$ (either 0 or 1.)

(though see 'fiducial inference')

Sidetrack: Coverage θ ← nature chooses & keeps fixed when generating data



Above 4/5 intervals trap the true value (empirical coverage = 0.8)

Coverage checks

- given a method, you can play the role of nature to check performance.
 - choose a θ , generate data from $P(Y; \theta)$, see how often interval traps true value.
 - get coverage for each choice of θ
 - usually want uniform coverage, eg 95% coverage for all θ
 - can do for selection or prove mathematically for all.

Simplest acceptable fit models

The previous CI procedure gives us a set of non-rejected (compatible) models for given data

↳ 'Lack of uniqueness solved' by accepting a set of models

We might stop there! But can also ask for a single representative

→ (in general there will be no 'most complicated' model consistent with the data (unbounded complexity))

→ there often is a 'simplest' (in the sense of our regularisation) model though!

↳ gives a lower bound on complexity (Donoho)

Popper!
Also Vapnik...
{
↳ simple models = falsifiable
↳ complex models = non-falsifiable, though may be true!
↳ (can be ruled out)

Procedure:

- Given m data points
- Find 95% upper limit for χ_m distribution (max misfit)
- Find min complexity (regularisation) model fitting no worse than above

Can also eg use 50% (median) as a more 'central' point estimate

↳ more typically used approach.

↳ Convenient fact: for large m , the median of χ_m is approx m .

→ choose $\|y^{obs} - X\theta\|^2 \approx \sigma^2 \cdot m$
for 'median' discrep. model.

Discrepancy approaches

Pros

- statistically well-justified } as interval
- simple interpretation } method

Cons

- need to know σ or noise model in general
 - tends to oversmooth when used as a point estimate (remember it's a lower bound!)
- Even when taking median tho.

Example: polynomial (x^{19}) regression, known noise level ($\sigma=1$)

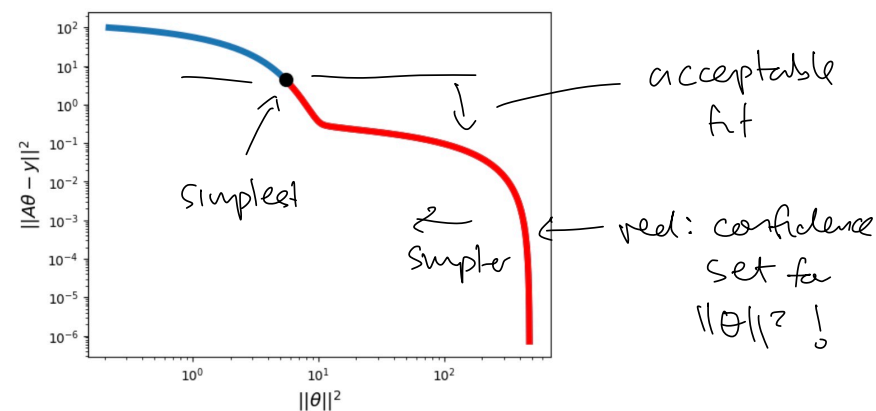
- Tikhonov $\lambda \|\theta\|^2$
- loop over each λ , calc norms
- Then:

median

```
from scipy.stats import chi2
delta = sigma**2*chi2.ppf(q=0.5,df=len(yp_obs))
indices = data_norms**2 <= delta
data_norms_delta = data_norms[indices]
theta_norms_delta = theta_norms[indices]
nreg = len(data_norms_delta)

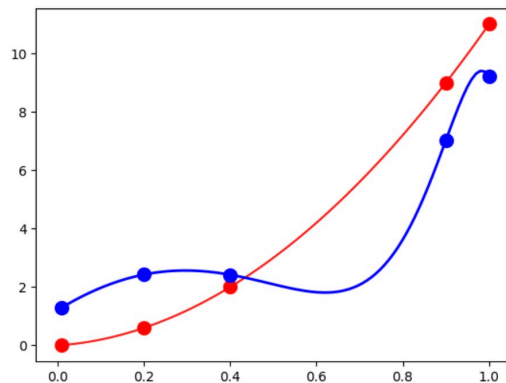
plt.loglog(theta_norms**2, data_norms**2, linewidth=5)
plt.loglog(theta_norms_delta**2, data_norms_delta**2, 'r', linewidth=5)
plt.loglog(theta_norms[nreg]**2, data_norms[nreg]**2, marker='o', markersize=10, linewidth=5, color='k')

plt.ylabel(r'$||A\theta - y||^2$', fontsize=14)
plt.xlabel(r'$||\theta||^2$', fontsize=14)
plt.show()
```

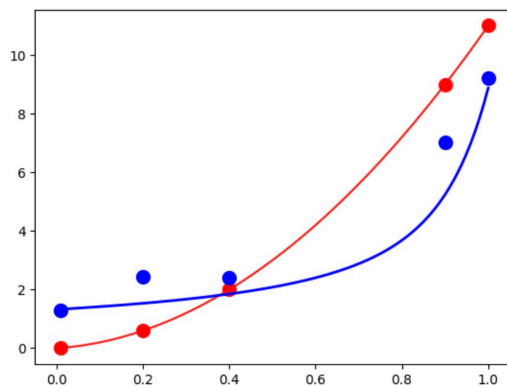


Note: even median quite oversmoothed as point estimate

Example cont' 1



pseudo
inverse
sol
(blue)



discrepancy
median
(over
smoothed
but treat
as
lower
bound)

Cross-validation & prediction

Cross-validation & its many variants are also widely used to choose regularisation parameters in inverse problems, statistics & machine learning

As before we aim to 'correct' the naive approach of just considering the models that just fit the given — here, 'training' — data

Why? Firstly, $\|y^{obs} - x\theta^{fit}\|^2$, where θ^{fit} is the model from minimising $\|y^{obs} - x\theta\|^2$ over θ , is known/can be shown to be a biased estimate of

$$\boxed{Err = E[\|y - x\theta^{true}\|^2]}$$

(can easily
generalise
to
nonlinear)

which is the expected prediction error,

where θ^{true} is fixed, y is randomly drawn from the distribution indexed by θ^{true} , x is sometimes random, typically fixed, i.e. the expected error for repeatedly predicting new samples at given x values using the true model.

Prediction cont'd

Why care about prediction?

1. Intrinsically reasonable thing to care about!

2. It can be shown that θ^{true} minimises the expected error for samples drawn from the distribution associated with θ^{true} - as it should be! i.e.

$$\begin{aligned} \arg \min_{\theta} \text{Err}(\theta) &= \mathbb{E}_{y|x; \theta^{\text{true}}} [(y - x\theta)^2] \\ &= \theta^{\text{true}} \end{aligned}$$

→ this is a form of statistical 'consistency' property.

[But] we don't know θ^{true} ! And we only have finite data available: $\mathcal{E}$ rather than $\mathbb{E}$

Bias & variance

Although $\|y^{\text{obs}} - x\theta^{\text{fit}}\|^2$ where θ^{fit} is a model fit using y^{obs} (perhaps via least sq.) is typically a biased estimate of pred. error, it may have low variance.

→ In statistics, risk = bias² + variance ← squared error loss
& we may accept some bias.

However, if the model family is very large, eg so that we can exactly fit any data, the bias of LS is very large
↳ always estimate error = 0!

→ hence we may want to construct an (approximately) unbiased estimate of the prediction error for any model, really any model fitting strategy

→ this motivates cross-validation

↳ more sophisticated versions may trade some bias & variance of pred. error estimate!

Recipe: leave-one-out cross validation

Given a sample $(x, y)_{i=1, \dots, m}$ & a λ -dependent rule for fitting a model to data, $\theta_{\lambda}^{\text{fit}}((x, y)_{i=1, \dots, m})$, where λ controls the allowed model complexity, e.g.

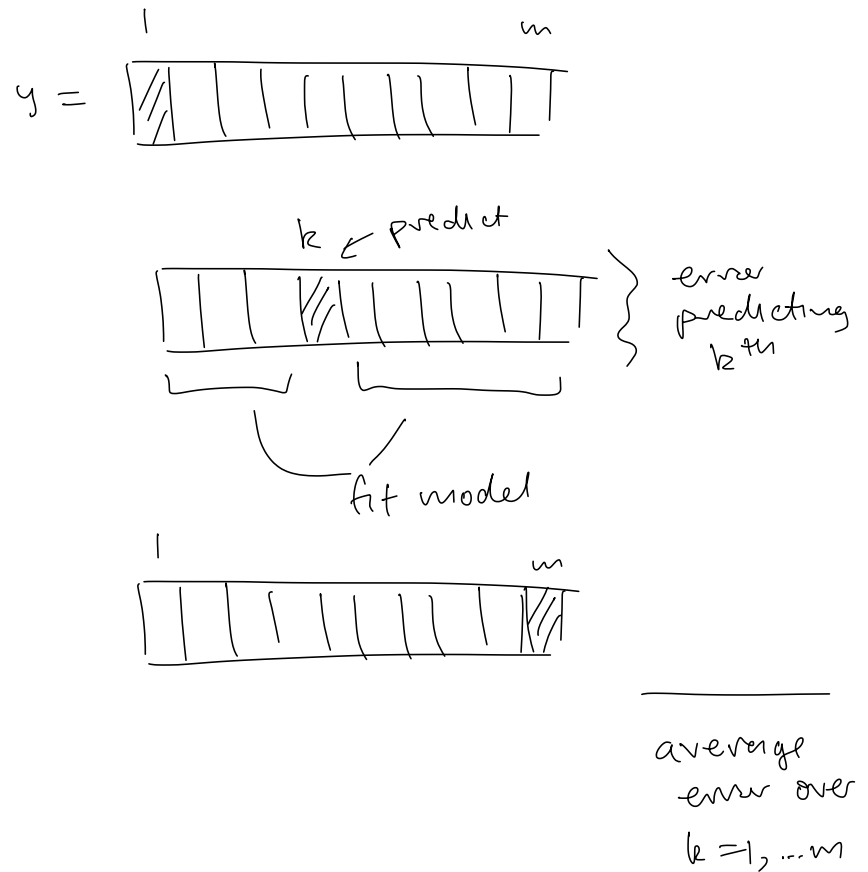
$$\theta_{\lambda}^{\text{fit}} = \arg \min_{\theta} \|y^{\text{obs}} - X\theta\|^2 + \lambda \|L\theta\|^2,$$

We define $\theta_{\lambda}^{\text{fit}, i \neq k}$ as the model fit on all data except the k^{th} pair $(x, y)_k$ & $X[k, :]$ as the observed forward map for the k^{th} observation & estimate the prediction error of the fitting rule by

$$\text{Err}(\theta_{\lambda}^{\text{fit}}) = \frac{1}{m} \sum_{k=1}^m (y_k^{\text{obs}} - X[k, :] \theta_{\lambda}^{\text{fit}, i \neq k})^2$$

i.e. the average error from fitting the model on the data with the k^{th} data point removed & predicting the k^{th} data point, for $k=1, \dots, m$ i.e. each data point in turn.

Picture:

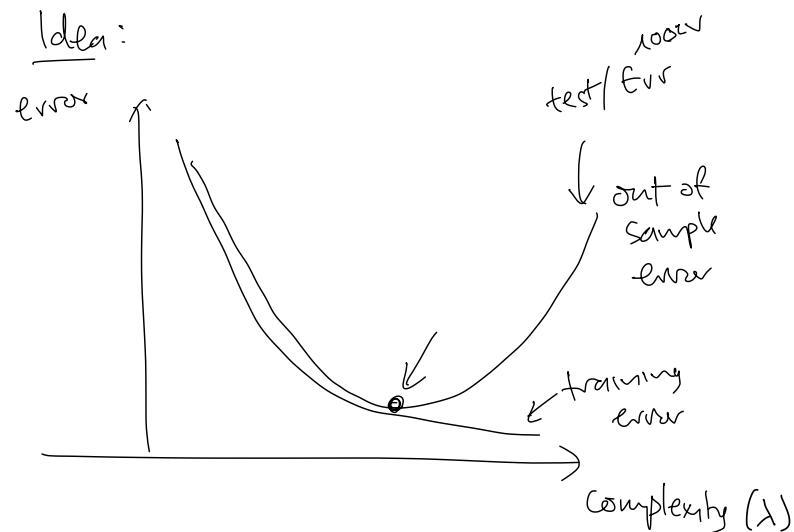


Estimating λ

Given our estimates for each λ ,
we choose the λ that minimises

$$\text{Err}(\theta_{\lambda}^{\text{fit}}) = \text{Err}(\lambda)$$

↑ ↑
only depends on λ



→ typically refit on whole dataset
with chosen λ to get final θ estimate
(tho can cause issues!)

Notes

- Simple & general, use for any fitting rule

- unbiased estimate of average
predictive error for $m-1$ data
points, $\approx$ average predictive error
for m data points

- averages over both inputs &
outputs → error rate for $\times$
random rather than fixed
(ie not error rate for this sample
but for rule in general)

- high variance estimate of prediction
error!

↳ Can improve with eg k-fold
cross validation or bootstrap
versions

k-fold: leave out $\approx m/k$ observations
at a time instead of one
→ lower variance, higher bias
(typically...)

- less comp. expensive — k fits per λ

Generalised cross validation: cheaper (for linear)

For linear models with linear regularisation

, it can be written

$$y^{\text{pred}} = X X_{\lambda}^{\#} y_{\text{obs}}, \quad X_{\lambda}^{\#}: \text{'regularised inverse'}$$

$$= R_{\lambda}^{\lambda} y_{\text{obs}} \quad \left. \begin{array}{l} \text{data space} \\ \text{pseudo projection} \end{array} \right\}$$

can show a reasonable approximation

to $\text{Err}^{\text{loocv}}(\lambda)$ is

$$\frac{1}{m} \frac{\|y^{\text{obs}} - X \theta_{\lambda}^{\text{fit}}\|^2}{\left(1 - \frac{\text{Trace}(X X_{\lambda}^{\#})}{m}\right)^2} = \frac{1}{m} \frac{\|y^{\text{obs}} - X \theta_{\lambda}^{\text{fit}}\|^2}{\left(1 - \frac{\text{Trace}(R_{\lambda})}{m}\right)^2}$$

correction factor
like $\text{eg } \sigma^2 = \frac{1}{n-1} \sum (y_i - \bar{y})^2$
for unbiased variance!

[Taylor series approx. tends to $\approx$ AIC]

See Aster et al Inverse Problems
Hastie et al. The elements of statistical
learning.

Example: CV attempt

```
thetas = np.zeros((len(alphav), len(thetap_true)))
theta_norms = np.zeros((len(alphav), len(yp_obs)))
data_norms = np.zeros((len(alphav), len(yp_obs)))
L = np.eye(len(thetap_true))
indices = np.arange(0, len(yp_obs), dtype=int)

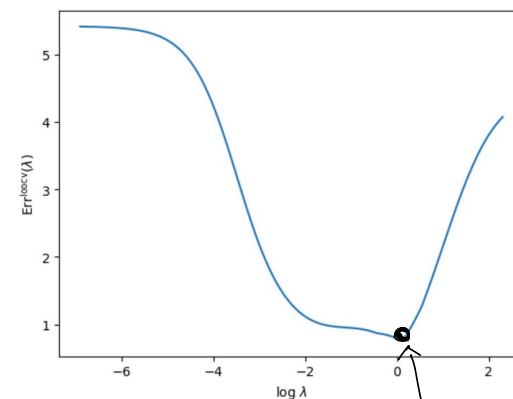
for i, alphai in enumerate(alphav):
    for k in range(0, len(yp_obs)):
        # indices
        keep_indices = indices[indices != k]

        # augment
        A_poly_aug_k = np.vstack([A_poly_obs[keep_indices, :], alphai*L])
        yp_aug_k = np.hstack([yp_obs[keep_indices], np.zeros(L.shape[0])])

        # invert
        A_poly_aug_k_pinv = np.linalg.pinv(A_poly_aug_k)
        theta_k = np.dot(A_poly_aug_k_pinv, yp_aug_k)

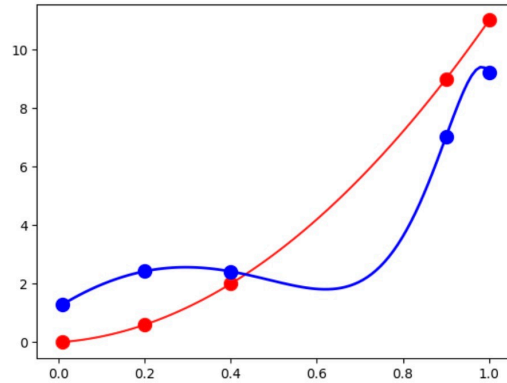
        # calc norms: note prediction on k!
        theta_norms[i, k] = np.linalg.norm(L@theta_k, 2)
        data_norms[i, k] = np.linalg.norm(yp_obs[k] - np.dot(A_poly_obs[k, :], theta_k))

# average data errors over k for each lambda
data_norms_cv = np.mean(data_norms, axis=1)
```

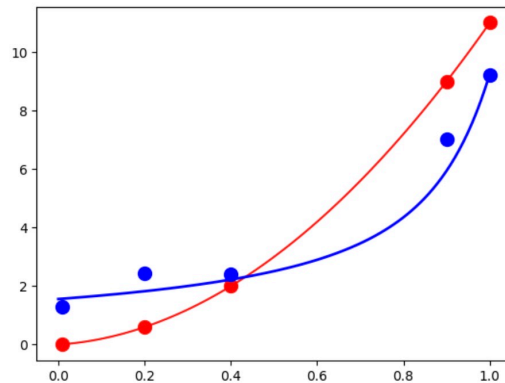


$\min \lambda \approx 1$

Example cont'd



no
regularization
(min norm
 A^+)



Tikhonov
+
CV

Further reading

- o Aster et al Inverse Problems
- o Hastie et al The elements of Statistical learning
- o Efron (1983) 'Estimating the error rate of a prediction rule'
- o Bates et al (2024) 'Cross-validation: what does it estimate & how well does it do it?'

Appendix : Double descent??

Reconciling modern machine-learning practice and the classical bias-variance trade-off

Mikhail Belkin^{a,b,1}, Daniel Hsu^a, Siyuan Ma^a, and Soumik Mandal^a

^aDepartment of Computer Science and Engineering, The Ohio State University, Columbus, OH 43210; ^bDepartment of Statistics, The Ohio State University, Columbus, OH 43210; and ¹Computer Science Department and Data Science Institute, Columbia University, New York, NY 10027

Edited by Peter J. Bickel, University of California, Berkeley, CA, and approved July 2, 2019 (received for review February 21, 2019)

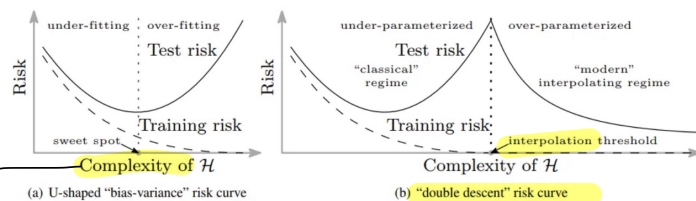


Figure 1: Curves for training risk (dashed line) and test risk (solid line). (a) The classical U-shaped risk curve arising from the bias-variance trade-off. (b) The double descent risk curve, which incorporates the U-shaped risk curve (i.e., the "classical" regime) together with the observed behavior from using high complexity function classes (i.e., the "modern" interpolating regime), separated by the interpolation threshold. The predictors to the right of the interpolation threshold have zero training risk.

- Some ML researchers have claimed the 'bias-variance' U curves of prediction risk are outdated
- The key issue in my opinion is distinguishing between:
 - dimension of model/data space } varying dim gives double descent
 - 'complexity', 'norm' etc of models in a space of given dimension } varying norm for fixed dim $\rightarrow$ no double descent
- ie: 'what is the x axis?'

Example: splines fit with pseudo inverse

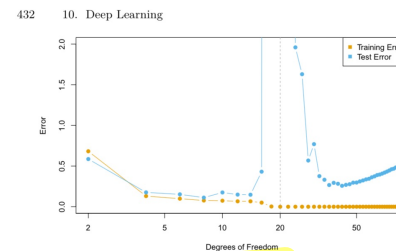


FIGURE 10.20. Double descent phenomenon, illustrated using error plots for a one-dimensional natural spline example. The horizontal axis refers to the number of spline basis functions on the log scale. The training error hits zero when the degrees of freedom coincides with the sample size $n = 20$, the "interpolation threshold", and remains zero thereafter. The test error increases dramatically at this threshold, but then descends again to a reasonable value before finally increasing again.

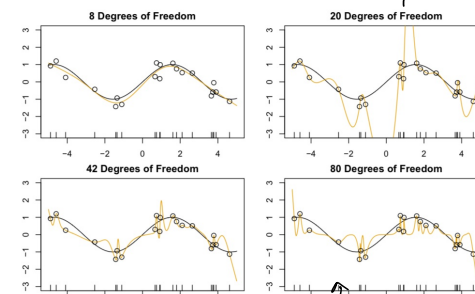


FIGURE 10.21. Fitted functions $\hat{f}_d(X)$ (orange), true function $f(X)$ (black) and the observed 20 training data points. A different value of d (degrees of freedom) is used in each panel. For $d \geq 20$ the orange curves all interpolate the training points, and hence the training error is zero.

dimension!

Some points are worth emphasizing:

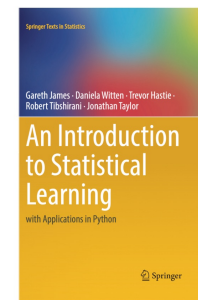
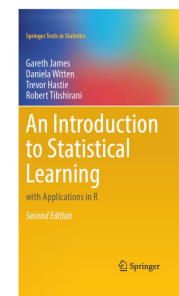
- The double-descent phenomenon does not contradict the bias-variance trade-off, as presented in Section 2.2.2. Rather, the double-descent curve seen in the right-hand side of Figure 10.20 is a consequence of the fact that the x-axis displays the number of spline basis functions used, which does not properly capture the true "flexibility" of models that interpolate the training data. Stated another way, in this example, the minimum-norm natural spline with $d = 42$ has lower variance than the natural spline with $d = 20$.

higher dimension but less complex!

dim

norm

from:



Think:

$$\min_{x \in \mathbb{R}^d} \|x\|^2 \text{ s.t. } Ax = b$$

$d \uparrow$ means bigger space to find smaller soln!